

技术报告

统一优化系统的 API Surface 收敛与策略编排说明

Status: Draft。本文以设计提案方式描述一个统一排序与重平衡层。目标不是追加 quick hack，而是在现有能力边界内收敛 design space，降低长期维护成本，并保持 backwards compatibility。

1. Motivation

当前能力由多个独立策略演进而来：质量过滤、相似度匹配、多样性控制、任务感知、历史反馈、对比表示和自洽验证。这些能力单独看都合理，但如果继续以布尔开关横向扩展，API surface 会持续变宽，调用路径会变得难以解释。统一优化系统的动机，是把分散策略抽象成一个可审阅的 ranking pipeline。

问题	现象	设计影响
API surface 扩张	调用参数持续增加	ergonomics 下降，误用概率增加
策略优先级隐式化	不同分支互相短路	结果难以复现和解释
共享样本节奏不稳定	部分任务被注入过多外部示例	引入负迁移风险
反馈信号分散	奖励、质量、相似度分别维护	长期演进成本上升

2. Guide-Level Explanation

对使用者而言，统一层只暴露一个概念：候选样本进入排序器，排序器根据任务配置合并有效信号，再通过重平衡规则控制共享样本比例。使用者不需要理解每个子策略的历史来源，只需要知道输入、输出和约束。

阶段	输入	输出	用户可见行为
过滤	原始候选	可用候选	低质量样本减少
融合	多路评分信号	统一分数	排序结果更稳定
重平衡	当前任务与共享池	最终示例集	共享注入比例可控
回退	极端空候选	基础排序结果	保持兼容而非失败

这不是新的训练器，也不是 breaking change。它是一个 orchestration layer，把既有能力以更少的入口重新组织。从 semver 视角看，这应当作为兼容性增强，而不是改变调用契约的主版本变更。

3. Reference-Level Explanation

参考层面包含三个核心实体：任务配置、统一排序器、共享样本重平衡器。任务配置负责声明各信号权重；统一排序器负责把质量、相似度、任务专属权重、历史奖励等信号映射到同一分数空间；重平衡器负责控制当前任务样本与共享样本的比例。

组件	责任边界	不负责的事项
任务配置	维护权重、阈值、共享比例	不直接读取模型输出
统一排序器	融合多路信号并排序	不修改原始样本内容
重平衡器	控制最终示例组成	不重新定义任务语义
兼容回退	在候选不足时保持可用	不掩盖配置错误

3.1 Scoring Contract

每个候选样本都被视为一个具有局部特征的实体。排序器只消费可解释信号，并避免把不可追踪的模型内部状态塞入分数。这样做的代价是上限受启发式规则限制；收益是 prior art 可复用、调试成本低、实现路径透明。

4. Drawbacks

该设计有明确代价。首先，统一排序器会增加单个函数的概念密度。其次，权重集中之后，局部调整可能影响多个任务。最后，如果没有足够的评测反馈，统一层可能只是把多个启发式规则包成一个更漂亮的抽象。

Drawback	Why it matters	Mitigation
配置耦合	一个权重影响多任务	按任务维护独立配置并保留默认值
调试路径变长	分数来源更多	输出可解释中间分数
抽象过度	统一层可能遮蔽简单问题	保留基础选择器作为回退
性能开销	候选排序信号增加	只对候选池做轻量特征计算

5. Rationale and Alternatives

被否决的替代方案一，是继续保留多个分支并按开关组合。它短期最省事，但长期会造成不可预测的优先级冲突。被否决的替代方案二，是引入集中训练器。它理论上更优雅，但会改变依赖边界，并带来明显的验证成本。

Alternative	Reason rejected
多分支并行开关	API surface 继续变宽，策略交互不可审阅
集中训练器	超出当前依赖约束，且引入新的数据与训练成本
仅保留单一相似度	实现简单但丢失质量、反馈和任务特化信号

6. Backwards Compatibility

兼容性原则是：默认路径可回退，既有输入格式不改变，输出仍是示例列表而不是新的复杂对象。如果统一层不可用，系统应回退到基础排序，而不是要求调用方理解内部状态。这一点避免了隐式 breaking change。

兼容项	约束	结果
输入数据	不改变样本字段语义	旧数据可继续使用
调用入口	保留既有选择函数	迁移成本低
输出结构	仍返回排序后的示例	下游无需改造
失败策略	候选不足时回退	避免运行期中断

7. Prior Art

该设计借鉴了前序工作中的动态自适应、跨任务共享、对比表示、强化反馈、分层组织和代码生成专项优化。与 prior art 的差异不在于发明一个全新的 scoring trick，而在于把这些 trick 放进一个可维护的 design space。

8. Unresolved Questions

仍需讨论三个问题。第一，任务权重是否应该进一步图谱化，而不是静态配置。第二，共享样本重平衡是否会压制极端任务的高价值样本。第三，面向代码生成任务是否应该拆出更小的 API surface，以避免通用排序器过度承担专项责任。

Question	Possible direction	Risk
权重是否图谱化	引入任务关系矩阵	解释成本上升
重平衡是否过强	按任务动态调节共享比例	评测波动变大
专项任务是否拆分	提供窄接口	抽象边界再次分裂

Conclusion: 推荐将统一优化层作为兼容性增强推进，但不建议把它包装成最终架构。它更像一个稳定的 RFC 起点：先收敛 API surface，再根据评测结果决定哪些策略值得晋升为长期接口。